

SQL Cheatsheet

Complete Beginner's Guide · DBMS Lab Ready

1. CREATE TABLE

Creates a new table in the database. Run this **FIRST** before inserting data.

```
CREATE TABLE Student(  
    student_id INT PRIMARY KEY,  
    name VARCHAR(50),  
    age INT,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```

Data Type	Meaning	Example
INT	Whole number	age INT
VARCHAR(n)	Text up to n characters	name VARCHAR(50)
FLOAT	Decimal number	marks FLOAT
DATE	Date value	dob DATE

■ Tables with **FOREIGN KEY** must be created **AFTER** the referenced table.

2. INSERT INTO

Adds rows of data into the table.

```
-- Insert all columns (values in same order as table)  
INSERT INTO Student VALUES (1, 'Sinchana', 20, 1);  
  
-- Insert specific columns only  
INSERT INTO Student (student_id, name) VALUES (2, 'Riya');  
  
-- Insert multiple rows at once  
INSERT INTO Student VALUES  
(3, 'Ananya', 21, 1),  
(4, 'Rahul', 22, 2);
```

■ Text → single quotes 'Sinchana' | Numbers → no quotes 20

3. SELECT

Fetches/reads data from the table. Most used command in SQL!

```
-- All columns  
SELECT * FROM Student;  
  
-- Specific columns
```

```

SELECT name, age FROM Student;

-- Rename columns in output (AS = alias)
SELECT name AS Student_Name, age AS Age FROM Student;

-- Remove duplicate rows
SELECT DISTINCT age FROM Student;

```

4. WHERE — Filter Rows

Filters only the rows that match a condition.

```

SELECT * FROM Student WHERE age > 20;

SELECT * FROM Student WHERE name = 'Sinchana';

SELECT * FROM Student WHERE age >= 20 AND dept_id = 1;

SELECT * FROM Student WHERE age = 20 OR age = 21;

SELECT * FROM Student WHERE name LIKE 'S%'; -- starts with S

SELECT * FROM Student WHERE age BETWEEN 18 AND 22;

SELECT * FROM Student WHERE dept_id IN (1, 2, 3);

```

Operator	Meaning
=	Equal to
!= or <>	Not equal
> / <	Greater / Less than
BETWEEN a AND b	Range (inclusive)
LIKE 'S%'	Pattern match (% = any chars)
IN (1,2,3)	Match any value in list
IS NULL	Check for empty/missing value

5. UPDATE

Changes existing data in a table.

```

-- Update one column
UPDATE Student SET age = 21 WHERE student_id = 1;

-- Update multiple columns
UPDATE Student SET age = 22, name = 'Sinchana SV' WHERE student_id = 1;

```

■ ALWAYS use WHERE with UPDATE. Without it → every row changes!

6. DELETE

Removes rows from a table.

```

DELETE FROM Student WHERE student_id = 2;

DELETE FROM Student WHERE age < 18;

DELETE FROM Student; -- ■ removes ALL rows!

```

Command	What it does
---------	--------------

DELETE FROM t WHERE ...	Remove specific rows
TRUNCATE TABLE t	Remove ALL rows (fast)
DROP TABLE t	Delete entire table permanently

7. ORDER BY — Sort Results

Sorts the output rows.

```
SELECT * FROM Student ORDER BY age ASC; -- low to high

SELECT * FROM Student ORDER BY age DESC; -- high to low

SELECT * FROM Student ORDER BY name ASC, age DESC; -- sort by 2 cols
```

8. GROUP BY + Aggregate Functions

Groups rows and performs calculations per group.

```
-- Count students per department

SELECT dept_id, COUNT(*) AS total FROM Student GROUP BY dept_id;

-- Average age per dept

SELECT dept_id, AVG(age) AS avg_age FROM Student GROUP BY dept_id;

-- Max and Min age

SELECT MAX(age), MIN(age) FROM Student;

-- Sum (useful for marks, salary etc.)

SELECT dept_id, SUM(age) FROM Student GROUP BY dept_id;

-- HAVING = WHERE for grouped results

SELECT dept_id, COUNT(*) FROM Student

GROUP BY dept_id HAVING COUNT(*) > 2;
```

Function	What it does
COUNT(*)	Count total rows
SUM(col)	Add up all values
AVG(col)	Average value
MAX(col)	Largest value
MIN(col)	Smallest value

9. JOIN — Combine Two Tables

Links rows from two tables using a common column.

```
-- INNER JOIN: only matching rows from both tables

SELECT s.name, d.dept_name

FROM Student s

JOIN Department d ON s.dept_id = d.dept_id;

-- LEFT JOIN: all rows from left table + matching from right
```

```

SELECT s.name, c.course_name

FROM Student s

LEFT JOIN Enrollment e ON s.student_id = e.student_id

LEFT JOIN Course c ON e.course_id = c.course_id;

-- 3-table JOIN

SELECT s.name, c.course_name, e.grade

FROM Enrollment e

JOIN Student s ON e.student_id = s.student_id

JOIN Course c ON e.course_id = c.course_id;

```

JOIN Type	Returns
INNER JOIN	Only rows that match in BOTH tables
LEFT JOIN	All rows from left + matches from right (NULL if no match)
RIGHT JOIN	All rows from right + matches from left
FULL JOIN	All rows from both tables

10. ALTER TABLE — Modify Table Structure

Changes the structure of an existing table (not data).

```

-- Add a new column

ALTER TABLE Student ADD phone VARCHAR(15);

-- Delete a column

ALTER TABLE Student DROP COLUMN phone;

-- Rename a column (MySQL)

ALTER TABLE Student RENAME COLUMN name TO full_name;

-- Change data type

ALTER TABLE Student MODIFY age FLOAT;

```

11. Constraints — Rules for Columns

```

CREATE TABLE Student(

student_id INT PRIMARY KEY, -- unique + not null

name VARCHAR(50) NOT NULL, -- cannot be empty

email VARCHAR(50) UNIQUE, -- no duplicates

age INT DEFAULT 18, -- auto-fills 18 if not given

dept_id INT,

FOREIGN KEY (dept_id) REFERENCES Department(dept_id)

);

```

Constraint	Meaning
------------	---------

PRIMARY KEY	Unique ID for each row, cannot be NULL
NOT NULL	Column must always have a value
UNIQUE	No two rows can have same value
DEFAULT val	Auto-fills value if none given
FOREIGN KEY	Links to another table's primary key
CHECK	Custom rule e.g. CHECK(age >= 18)

12. Subquery — Query Inside a Query

Use the result of one query inside another.

```
-- Students older than average age

SELECT name FROM Student

WHERE age > (SELECT AVG(age) FROM Student);

-- Students enrolled in 'DBMS'

SELECT name FROM Student

WHERE student_id IN (

SELECT student_id FROM Enrollment

WHERE course_id = (SELECT course_id FROM Course WHERE course_name = 'DBMS')

);
```

13. Query Writing Order (Remember This!)

```
SELECT col1, col2, AGG_FUNC(col3) -- WHAT to show

FROM table_name -- WHERE from

JOIN other_table ON condition -- combine tables

WHERE condition -- filter rows

GROUP BY col1 -- group for aggregates

HAVING aggregate_condition -- filter groups

ORDER BY col1 DESC -- sort

LIMIT 10; -- max rows to show
```

■ Write in this order every time and you'll never get confused!